

ESPECIFICAÇÃO ARQUITETURAL

Trabalho Prático 1 — Arquitetura de Software

Bruno Rodrigues dos Santos
Carlos Eduardo Carvalho Meneguette
Leandro de Araujo
Vinicius Andrade Michaki Hubner

2026

1. Descrição da Aplicação

O sistema proposto é um Sistema de Gestão Acadêmica, desenvolvido para auxiliar instituições de ensino no controle de informações relacionadas a alunos, professores, disciplinas, notas e frequência.

O problema central abordado consiste na dificuldade de organização e controle de dados acadêmicos. Muitas instituições ainda dependem de processos manuais ou de sistemas pouco integrados, o que resulta em inconsistência de informações, dificuldade de acesso e baixa eficiência na gestão.

1.1 Objetivos do Sistema

- Facilitar o registro e o acompanhamento de notas e frequência dos alunos.
- Melhorar a organização e o controle das informações acadêmicas.
- Garantir acesso confiável aos dados por parte de alunos e professores.

1.2 Usuários Principais

Aluno: consulta de notas, frequência e desempenho acadêmico.

Professor: lançamento de notas, registro de faltas e acompanhamento de turmas.

Administrador: gerenciamento de usuários, disciplinas e estrutura do sistema.

2. Arquitetura Escolhida

A arquitetura adotada para o desenvolvimento do sistema é a Clean Architecture (Arquitetura Limpa). Essa abordagem organiza a aplicação em camadas concêntricas e independentes, orientadas pelas regras de negócio, garantindo que o núcleo do sistema não dependa de tecnologias ou frameworks externos.

3. Justificativa da Escolha Arquitetural

A Clean Architecture foi escolhida por se adequar às características específicas de um sistema acadêmico, no qual as regras de negócio são numerosas, detalhadas e sujeitas a alterações frequentes.

As principais motivações para essa escolha são:

- Isolamento das regras de negócio: funcionalidades como cálculo de média, validação de frequência, matrícula em disciplinas e controle de pré-requisitos ficam no núcleo da aplicação, sem dependência de frameworks, banco de dados ou interface gráfica.
- Independência tecnológica: alterações em tecnologias externas — como troca de banco de dados, framework backend ou adição de uma interface mobile — não afetam o núcleo do sistema, reduzindo o custo de manutenção.

- Testabilidade: a separação entre camadas permite testar funcionalidades como aprovação, reprovação e matrícula de forma isolada, sem necessidade de configurar infraestrutura completa.
- Organização e escalabilidade: a estrutura em camadas facilita a divisão de responsabilidades entre os membros da equipe e suporta a adição de novas funcionalidades sem comprometer as existentes.

4. Requisitos e Escopo Inicial

4.1 Funcionalidades Previstas

- Cadastro de alunos, professores e administradores.
- Autenticação de usuários com controle de acesso por perfil.
- Cadastro e gerenciamento de disciplinas.
- Matrícula de alunos em disciplinas.
- Lançamento de notas pelos professores.
- Registro de frequência (faltas por disciplina).
- Consulta de notas e frequência pelos alunos.
- Cálculo automático de média final.
- Identificação de situação acadêmica (aprovado ou reprovado).

4.2 Escopo Mínimo para o TP2

Para a próxima etapa do projeto, serão implementadas as seguintes funcionalidades essenciais:

- Autenticação de usuários com diferenciação de perfis (aluno, professor e administrador).
- Cadastro de alunos e disciplinas.
- Matrícula de alunos em disciplinas.
- Lançamento de notas pelos professores.
- Registro de faltas por disciplina.
- Consulta de desempenho acadêmico (notas e frequência) pelos alunos.
- Cálculo de média final e definição da situação acadêmica.
- Persistência de dados em banco de dados relacional (SQLite).
- Interface de acesso via interface web básica.

Funcionalidades como relatórios avançados, notificações automatizadas e integrações com sistemas externos estão fora do escopo desta etapa e poderão ser incorporadas em versões futuras.

5. Decomposição do Sistema

O sistema é estruturado em quatro camadas concêntricas, seguindo os princípios da Clean Architecture. Cada camada possui responsabilidades bem definidas, e a dependência entre

elas flui sempre de fora para dentro: as camadas externas dependem das internas, nunca o contrário.

5.1 Entidades (Entities)

Representam as regras de negócio centrais e os objetos principais do domínio acadêmico. São independentes de qualquer tecnologia ou framework.

- Aluno – dados pessoais, matrícula e situação acadêmica.
- Professor – identificação e vínculo com disciplinas.
- Disciplina – código, nome, carga horária e pré-requisitos.
- Nota – valor numérico, tipo de avaliação e vínculo com aluno e disciplina.
- Frequencia – registro de presença ou falta por aula/período.

5.2 Casos de Uso (Use Cases)

Implementam a lógica da aplicação, coordenando entidades e repositórios para executar as operações concretas do sistema.

- CadastrarAluno – valida e persiste um novo aluno no sistema.
- CadastrarDisciplina – registra uma nova disciplina com suas regras.
- MatricularAluno – associa um aluno a uma disciplina, respeitando pré-requisitos.
- LancarNota – registra avaliação de um aluno em determinada disciplina.
- RegistrarFalta – computa ausência de um aluno em uma aula.
- CalcularMedia – aplica a regra de cálculo de média final.
- ConsultarDesempenho – retorna notas e frequência consolidadas por aluno.
- AutenticarUsuario – valida credenciais e retorna o perfil de acesso.

5.3 Adaptadores de Interface (Interface Adapters)

Realizam a comunicação entre o núcleo do sistema e os mecanismos externos, traduzindo dados e chamadas entre as camadas.

- Controllers – recebem requisições externas (HTTP) e delegam aos casos de uso.
- Presenters – formatam os dados de saída para o formato esperado pela interface.
- Repositórios (interfaces) – definem contratos de acesso a dados, implementados na camada de infraestrutura.

5.4 Frameworks e Infraestrutura (Frameworks & Drivers)

Contêm as implementações concretas das tecnologias externas. É a única camada que pode ser substituída sem impacto nas camadas internas.

- Banco de dados – implementação dos repositórios com SQLite.
- Interface web – camada de apresentação acessada pelo navegador.

- Framework backend – responsável pelo roteamento e gerenciamento de requisições HTTP.

6. Relação com POO e SOLID

6.1 Princípios de Orientação a Objetos

Encapsulamento: as entidades (Aluno, Nota, Frequencia) agrupam dados e comportamentos, expondo apenas as operações necessárias.

Abstração: repositórios e serviços são definidos como interfaces, permitindo que a camada de casos de uso opere sem conhecer as implementações concretas.

Composição: casos de uso são construídos pela combinação de entidades, repositórios e serviços auxiliares, evitando herança excessiva.

6.2 Princípios SOLID

Single Responsibility Principle (SRP): cada classe possui uma única razão para ser alterada. Entidades cuidam das regras de domínio; casos de uso, da lógica de aplicação; repositórios, da persistência.

Open/Closed Principle (OCP): o sistema pode ser estendido — por exemplo, com novos tipos de avaliação — sem necessidade de modificar código existente.

Liskov Substitution Principle (LSP): as implementações concretas dos repositórios respeitam os contratos definidos pelas interfaces, podendo ser substituídas sem alterar o comportamento esperado.

Interface Segregation Principle (ISP): interfaces são definidas de forma específica para cada contexto, evitando dependências desnecessárias entre componentes.

Dependency Inversion Principle (DIP): as camadas internas dependem de abstrações (interfaces), não de implementações concretas. A injeção de dependências garante esse desacoplamento em tempo de execução.

7. Padrões de Projeto

Os padrões a seguir foram selecionados por sua compatibilidade direta com a Clean Architecture, contribuindo para a modularidade, testabilidade e manutenibilidade do sistema.

Padrão	Função no Sistema	Camada de Aplicação
Repository	Abstrai o acesso ao banco de dados, desacoplando a lógica de persistência das regras de negócio.	Interface Adapters / Infraestrutura
Use Case (Application Service)	Organiza e executa a lógica de aplicação, coordenando entidades e repositórios para cada operação.	Use Cases
Dependency Injection	Injeta implementações concretas nas dependências abstratas, garantindo baixo acoplamento entre as camadas.	Todas as camadas

Padrão	Função no Sistema	Camada de Aplicação
DTO (Data Transfer Object)	Transporta dados entre camadas sem expor entidades de domínio às camadas externas.	Interface Adapters

8. Diagrama Arquitetural

O diagrama a seguir ilustra a organização em camadas da Clean Architecture aplicada ao Sistema de Gestão Acadêmica. As dependências fluem de fora para dentro: as camadas externas (Frameworks e Interface Adapters) dependem das internas (Use Cases e Entities), nunca o contrário.

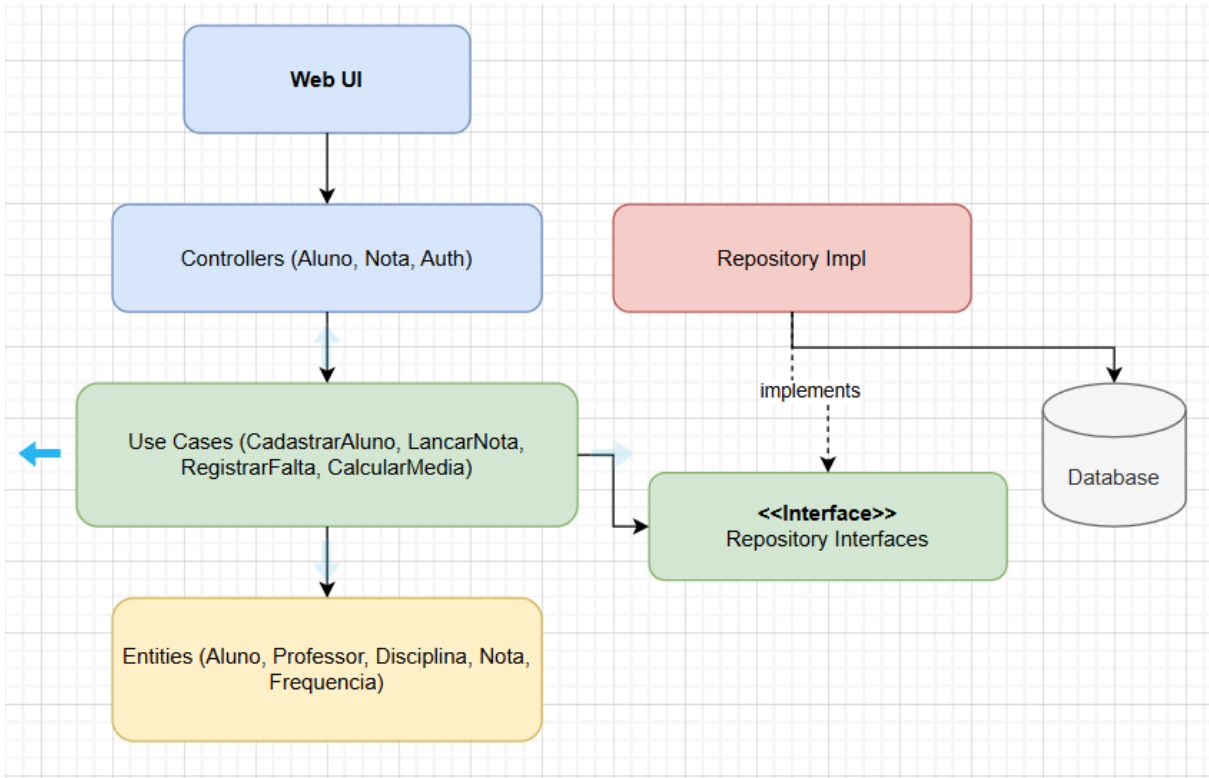


Figura 1 — Diagrama arquitetural do Sistema de Gestão Acadêmica (Clean Architecture).

9. Requisitos Arquiteturais da Solução

Esta seção analisa como a Clean Architecture atende aos principais requisitos arquiteturais do Sistema de Gestão Acadêmica. Para cada requisito, são avaliadas sua relevância para o projeto, onde se manifesta no sistema, como a arquitetura o favorece ou limita e como deve estar repre

Requisito Arquitetural	É importante para o sistema? Por quê?	Onde aparece no sistema?	Como a arquitetura ajuda ou limita?	Como isso deve aparecer na solução?
------------------------	---------------------------------------	--------------------------	-------------------------------------	-------------------------------------

Manutenibilidade	Sim. As regras acadêmicas podem mudar com frequência, como cálculo de média, frequência e matrícula.	Regras de aprovação, notas, frequência e disciplinas.	A Clean Architecture facilita a manutenção ao separar regras de negócio das partes técnicas.	Separação em camadas independentes e organização clara do código.
Modularidade / Baixo acoplamento	Sim. O sistema possui funcionalidades diferentes que precisam evoluir sem afetar todo o projeto.	Módulos de matrícula, notas, histórico e usuários.	A arquitetura reduz dependência direta entre camadas.	Cada módulo deve possuir responsabilidades bem definidas.
Testabilidade	Sim. As regras acadêmicas precisam ser validadas com segurança.	Cálculo de média, aprovação e validação de matrícula.	O domínio pode ser testado sem depender de banco ou interface.	Testes unitários das regras de negócio.
Segurança	Sim. O sistema possui diferentes tipos de usuários e dados acadêmicos.	Login, permissões de professor, aluno e coordenação.	A arquitetura permite separar autenticação das regras de negócio.	Controle de acesso por perfil e autenticação de usuários.
Desempenho	Moderadamente importante. O sistema precisa responder de forma adequada para consultas e lançamentos.	Consulta de histórico, notas e disciplinas.	A arquitetura não melhora desempenho diretamente, mas mantém organização para otimizações futuras.	Consultas simples e estrutura organizada de acesso aos dados.
Escalabilidade	Parcialmente importante. O sistema pode crescer em funcionalidades futuramente.	Inclusão de novos módulos e funcionalidades acadêmicas.	A separação em camadas facilita expansão do sistema.	Estrutura preparada para novos módulos sem alterar o núcleo.

Disponibilidade / Confiabilidade	Sim. O sistema precisa manter consistência das informações acadêmicas.	Notas, frequência, histórico e matrícula.	O isolamento das regras reduz risco de erros em mudanças futuras.	Validação consistente das regras acadêmicas e persistência correta dos dados.
Integração / Interoperabilidade	Parcialmente importante. O sistema pode futuramente integrar APIs ou outros sistemas acadêmicos.	APIs externas, exportação de dados e autenticação.	A camada de infraestrutura facilita futuras integrações sem alterar o domínio.	APIs e serviços externos isolados da lógica de negócio.

10. Riscos Arquiteturais e Trade-offs da Solução

10.1 Riscos Arquiteturais

Esta seção identifica os principais riscos que podem comprometer a implementação, manutenção ou evolução do sistema caso não sejam tratados desde o início.

Risco arquitetural	Por que isso pode ser um problema?	Como o grupo pretende reduzir esse risco?
Regras acadêmicas fora do domínio	Regras de matrícula, média ou frequência podem acabar sendo implementadas nos controllers, dificultando manutenção futura.	Centralizar as regras acadêmicas na camada de domínio da aplicação.
Excesso de separação para um projeto acadêmico	A quantidade de camadas pode aumentar a complexidade inicial do sistema.	Utilizar apenas as camadas necessárias para o escopo do projeto acadêmico.

Dependência incorreta entre camadas	Controllers ou infraestrutura podem acabar acessando regras internas diretamente.	Seguir a estrutura da Clean Architecture e manter o fluxo de dependência voltado para o núcleo.
-------------------------------------	---	---

10.2 Trade-offs da Arquitetura Escolhida

Todo trade-off arquitetural representa uma decisão consciente: a Clean Architecture favorece determinados atributos de qualidade em detrimento de outros. A seguir, os cinco principais trade-offs identificados para este projeto.

Trade-off	O que a arquitetura favorece?	O que ela pode dificultar?	Como o grupo pretende lidar com essa limitação?
1. Separação das regras acadêmicas	Isolamento de regras como cálculo de média e frequência.	Estrutura inicial mais complexa.	Definir organização clara das camadas desde o início.
2. Independência do banco de dados	Possibilidade de trocar PostgreSQL ou MySQL sem alterar regras acadêmicas.	Necessidade de criar camada de acesso aos dados.	Manter repositórios simples e focados apenas em persistência.
3. Testes isolados do domínio	Validar matrícula, notas e aprovação sem interface gráfica.	Maior tempo inicial para estruturar testes.	Priorizar testes das funcionalidades acadêmicas principais.
4. Baixo acoplamento entre módulos	Facilita evolução de módulos como notas, histórico e disciplinas.	Maior quantidade de arquivos e separações.	Padronizar nomenclatura e estrutura do projeto.
5. Facilidade de expansão futura	Inclusão de novos módulos sem alterar o núcleo do sistema.	Pode parecer excessivo para funcionalidades simples.	Aplicar a arquitetura apenas nas partes centrais do sistema.